

บทที่ 3

การจัดการข้อมูลในจาวาสคริปต์ (Data Management in JavaScript)

เนื้อหา

- การจัดการข้อมูลใน JavaScript คืออะไร?
- อาร์เรย์ (Array) และเมธอดของอาร์เรย์ (map, filter, reduce, forEach) ใน JavaScript
- หลักการและความจำเป็นของอาร์เรย์
- อ็อบเจกต์ (Object) และการเข้าถึงข้อมูลใน JavaScript
- การใช้งาน Array + Object เป็นเทคนิคที่สำคัญมากในการพัฒนาเว็บแอปพลิเคชัน

การจัดการข้อมูลใน JavaScript สามารถทำได้อย่างยืดหยุ่นผ่านโครงสร้างข้อมูลหลักอย่าง อาร์เรย์ (Array) และ อ็อบเจกต์ (Object) โดยอาร์เรย์ใช้เก็บข้อมูลหลายค่าที่มีลำดับ เช่น [1, 2, 3] และมาพร้อมเมธอดที่ช่วยให้จัดการข้อมูลได้ง่ายขึ้น เช่น map() ใช้แปลงค่าทุกตัวในอาร์เรย์, filter() ใช้คัดกรองข้อมูลตามเงื่อนไข, reduce() ใช้รวมค่าทั้งหมดให้เป็นค่าเดียว เช่น ผลรวม, และ forEach() ใช้ในการวนลูปทำงานกับแต่ละค่าของอาร์เรย์ โดยไม่คืนค่าใหม่ ช่วยให้การประมวลผลข้อมูลทำได้รวดเร็วและกระชับ

สำหรับ อ็อบเจกต์ (Object) คือโครงสร้างข้อมูลที่เก็บค่าต่าง ๆ เป็นคู่ key-value เช่น {name: "John", age: 25} เราสามารถเข้าถึงหรือแก้ไขค่าภายในอ็อบเจกต์ได้ผ่าน dot notation (obj.name) หรือ bracket notation (obj["name"]) ขึ้นอยู่กับกรณี โดยอ็อบเจกต์เหมาะสำหรับจัดเก็บข้อมูลที่มีโครงสร้างซับซ้อน และสามารถซ้อนกันได้ (Nested Object) การใช้ array และ object ร่วมกันอย่างเหมาะสมจึงเป็นหัวใจสำคัญในการจัดการข้อมูลใน JavaScript อย่างมีประสิทธิภาพ.

การจัดการข้อมูลใน JavaScript คืออะไร?

การจัดการข้อมูลใน JavaScript หมายถึง การสร้าง, จัดเก็บ, ดึง, แก้ไข และลบข้อมูล ที่ใช้ในโปรแกรม ซึ่งสามารถอยู่ในรูปของตัวแปร, ออบเจกต์, อาร์เรย์ หรือโครงสร้างข้อมูลอื่นๆ JavaScript มีเครื่องมือและเทคนิคหลายอย่างที่จะช่วยให้การจัดการข้อมูลมีประสิทธิภาพ เช่น ตัวแปร, อาร์เรย์, ออบเจกต์, JSON, และการจัดการข้อมูลฝั่งเซิร์ฟเวอร์

☐ วิธีการจัดการข้อมูลใน JavaScript

1 ☒ ตัวแปร (Variables)

ใช้ var, let, const เพื่อเก็บข้อมูลที่สามารถเปลี่ยนแปลงได้ในโปรแกรม

2 ☐ ชนิดของข้อมูล (Data Types)

ข้อมูลใน JavaScript แบ่งเป็น 2 ประเภทหลัก

- **Primitive Types** เช่น string, number, boolean, null, undefined
- **Reference Types** เช่น array, object, function

3 ☐ อาร์เรย์ (Arrays)

ใช้เก็บข้อมูลหลายค่าในตัวแปรเดียว เช่น [1, 2, 3, 4]

4 ☐ วัตถุ (Objects)

ใช้เก็บข้อมูลในรูปแบบ key-value pair เช่น {name: "John", age: 30}

5 ☐ การดำเนินการกับข้อมูล (Operators)

ใช้ตัวดำเนินการ เช่น +, -, *, /, %, &&, ||, ! เพื่อคำนวณและเปรียบเทียบข้อมูล

6 ☐ การควบคุมโฟลว์ (Control Flow)

ใช้ if-else, switch-case, for, while, do-while เพื่อตัดสินใจและทำซ้ำ

7 ☐ ฟังก์ชัน (Functions)

ใช้ฟังก์ชันเพื่อแบ่งโค้ดออกเป็นส่วนย่อยๆ เช่น function greet(name) { return "Hello " + name; }

8 ☐ การจัดการ JSON (JavaScript Object Notation)

ใช้ JSON เพื่อแลกเปลี่ยนข้อมูล เช่น JSON.stringify() และ JSON.parse()

9 ☐ การจัดการข้อมูลฝั่งไคลเอนต์ (Client-Side Storage)

- **localStorage** และ **sessionStorage** ใช้เก็บข้อมูลบนเบราว์เซอร์
- **Cookies** ใช้เก็บข้อมูลขนาดเล็ก เช่น การจดจำผู้ใช้

☐ การเรียกใช้ข้อมูลจากเซิร์ฟเวอร์ (AJAX & Fetch API)

ใช้ fetch() หรือ XMLHttpRequest เพื่อรับและส่งข้อมูลจากเซิร์ฟเวอร์

☐ สรุป

การจัดการข้อมูลใน JavaScript มีหลายวิธี เช่น การใช้ตัวแปร, อาร์เรย์, วัตถุ, JSON, ฟังก์ชัน และการดึงข้อมูลจากเซิร์ฟเวอร์ นักพัฒนาควรเลือกวิธีที่เหมาะสมกับงานเพื่อเพิ่มประสิทธิภาพของโปรแกรม ☐

อาร์เรย์ (Array) และเมธอดของอาร์เรย์ (map, filter, reduce, forEach) ใน JavaScript

☐ อาร์เรย์ (Array) คืออะไร?

อาร์เรย์ (Array) เป็นโครงสร้างข้อมูลที่ใช้ในการเก็บ ค่าหลายค่า ไว้ในตัวแปรเดียว โดยสามารถเข้าถึงค่าแต่ละตัวได้ด้วย **index** ซึ่งเริ่มต้นที่ 0

☐ การประกาศอาร์เรย์

```
let numbers = [10, 20, 30, 40, 50]; // อาร์เรย์ตัวเลข
let fruits = ["Apple", "Banana", "Mango"]; // อาร์เรย์ข้อความ
let mixedArray = [1, "Hello", true, { name: "John" }]; // อาร์เรย์แบบผสม
```

☐ การเข้าถึงข้อมูลในอาร์เรย์

```
console.log(numbers[0]); // 10
console.log(fruits[1]); // Banana
```

☐ การเปลี่ยนค่าข้อมูลในอาร์เรย์

```
numbers[2] = 100;
console.log(numbers); // [10, 20, 100, 40, 50]
```

☐ เมธอดของอาร์เรย์ (Array Methods)

JavaScript มีเมธอดที่ช่วยให้เราสามารถจัดการอาร์เรย์ได้ง่ายขึ้น เช่น `map()`, `filter()`, `reduce()`, และ `forEach()`

1 ☐ `map()` – ใช้แปลงค่าภายในอาร์เรย์

`map()` ใช้ สร้างอาร์เรย์ใหม่ โดยใช้ฟังก์ชันที่กำหนดให้ทำงานกับแต่ละค่าของอาร์เรย์เดิม

☐ ตัวอย่าง: แปลงอาร์เรย์ตัวเลขให้เป็นค่าที่คูณด้วย 2

```
let numbers = [1, 2, 3, 4, 5];
let doubledNumbers = numbers.map(num => num * 2);
console.log(doubledNumbers); // [2, 4, 6, 8, 10]
```

☐ ตัวอย่าง: แปลงอาร์เรย์ของออบเจกต์

```
let users = [
  { name: "Alice", age: 25 },
  { name: "Bob", age: 30 }
];

let names = users.map(user => user.name);
console.log(names); // ["Alice", "Bob"]
```

2 ☐ `filter()` – ใช้กรองข้อมูลในอาร์เรย์

`filter()` ใช้ สร้างอาร์เรย์ใหม่ โดยเลือกเฉพาะค่าที่ตรงกับเงื่อนไขที่กำหนด

☐ ตัวอย่าง: กรองตัวเลขที่มากกว่า 10

```
let numbers = [5, 10, 15, 20, 25];
let greaterThanTen = numbers.filter(num => num > 10);
```

```
console.log(greaterThanTen); // [15, 20, 25]
```

☐ ตัวอย่าง: กรองคนที่อายุมากกว่า 18

```
let people = [  
  { name: "John", age: 17 },  
  { name: "Jane", age: 22 },  
  { name: "Mike", age: 30 }  
];  
  
let adults = people.filter(person => person.age >= 18);  
console.log(adults);  
// [{ name: "Jane", age: 22 }, { name: "Mike", age: 30 }]
```

3 ☐ **reduce()** – ใช้รวมค่าทั้งหมดในอาร์เรย์

reduce() ใช้ รวมค่าทั้งหมดในอาร์เรย์ ให้อยู่ในค่าเดียว โดยใช้ฟังก์ชันที่กำหนด

☐ ตัวอย่าง: หาผลรวมของอาร์เรย์ตัวเลข

```
let numbers = [1, 2, 3, 4, 5];  
let sum = numbers.reduce((total, num) => total + num, 0);  
console.log(sum); // 15
```

☐ ตัวอย่าง: รวมราคาสินค้าในตะกร้าสินค้า

```
let cart = [  
  { name: "Notebook", price: 1500 },  
  { name: "Mouse", price: 500 },  
  { name: "Keyboard", price: 1000 }  
];  
  
let totalPrice = cart.reduce((total, item) => total + item.price, 0);  
console.log(totalPrice); // 3000
```

4 ☐ **forEach()** – ใช้ทำซ้ำแต่ละค่าของอาร์เรย์

forEach() ใช้ วนลูปทำงานกับแต่ละค่าของอาร์เรย์ โดยไม่สร้างอาร์เรย์ใหม่

☐ ตัวอย่าง: แสดงค่าทั้งหมดในอาร์เรย์

```
let numbers = [1, 2, 3, 4, 5];  
numbers.forEach(num => console.log(num));  
// 1
```

// 2

// 3

// 4

// 5

☐ ตัวอย่าง: แสดงข้อมูลจากอาร์เรย์ของออบเจกต์

let users = [

{ name: "Alice", age: 25 },

{ name: "Bob", age: 30 }

];

users.forEach(user => console.log(user.name));

// Alice

// Bob

☐ เปรียบเทียบเมธอดของอาร์เรย์

เมธอด	ใช้ทำอะไร	เปลี่ยนแปลงอาร์เรย์เดิมหรือไม่?	ผลลัพธ์เป็นอะไร?
map()	สร้างอาร์เรย์ใหม่จากอาร์เรย์เดิมโดยเปลี่ยนค่าทุกตัว	☐ ไม่เปลี่ยนอาร์เรย์เดิม	อาร์เรย์ใหม่
filter()	กรองข้อมูลในอาร์เรย์ตามเงื่อนไขที่กำหนด	☐ ไม่เปลี่ยนอาร์เรย์เดิม	อาร์เรย์ใหม่
reduce()	รวมค่าทั้งหมดในอาร์เรย์เป็นค่าเดียว	☐ ไม่เปลี่ยนอาร์เรย์เดิม	ค่าเดียว (Number, Object, Array ฯลฯ)
forEach()	วนลูปทำงานกับแต่ละค่าของอาร์เรย์	☐ ไม่เปลี่ยนอาร์เรย์เดิม	ไม่มีค่ารีเทิร์น (undefined)

☐ สรุป☐ อาร์เรย์ (Array) คือโครงสร้างข้อมูลที่ใช้เก็บค่าหลายค่าภายในตัวแปรเดียว☐ map() ใช้แปลงค่าภายในอาร์เรย์เป็นอาร์เรย์ใหม่☐ filter() ใช้คัดกรองข้อมูลที่ตรงเงื่อนไขในอาร์เรย์☐ reduce() ใช้รวมค่าทั้งหมดในอาร์เรย์ให้อยู่ในค่าเดียว☐ forEach() ใช้ทำงานกับแต่ละค่าของอาร์เรย์โดยไม่สร้างอาร์เรย์ใหม่

หลักการและความจำเป็นของอาร์เรย์

1. ช่วยลดการใช้ตัวแปรซ้ำซ้อน

- ถ้าไม่มีอาร์เรย์ เราจะต้องสร้างตัวแปรหลายตัว เช่น
- `let num1 = 10, num2 = 20, num3 = 30;`
- แต่ใช้ อาร์เรย์ เราสามารถเก็บค่าทั้งหมดในตัวแปรเดียวได้
- `let numbers = [10, 20, 30];`

2. จัดการข้อมูลได้ง่ายขึ้น

- ใช้ ลูป เช่น `for`, `forEach()` เพื่อเข้าถึงข้อมูลทั้งหมดในอาร์เรย์
- มีเมธอด (`map()`, `filter()`, `reduce()`, `forEach()`) ที่ช่วยให้จัดการข้อมูลได้ง่ายขึ้น

☐ ไวยากรณ์ของอาร์เรย์

`let arrayName = [value1, value2, value3, ...];` // แบบที่ 1

`let arrayName = new Array(value1, value2, value3, ...);` // แบบที่ 2

☐ ตัวอย่าง

`let colors = ["Red", "Green", "Blue"];`

`console.log(colors[0]);` // Red

☐ เมธอดของอาร์เรย์ (Array Methods)

JavaScript มีเมธอดที่ช่วยให้เราสามารถจัดการอาร์เรย์ได้ง่ายขึ้น เช่น `map()`, `filter()`, `reduce()`, และ `forEach()`

1 ☐ `map()` – ใช้แปลงค่าภายในอาร์เรย์

`map()` ใช้ สร้างอาร์เรย์ใหม่ โดยใช้ฟังก์ชันที่กำหนดให้ทำงานกับแต่ละค่าของอาร์เรย์เดิม

☐ ไวยากรณ์

```
let newArray = oldArray.map((element, index, array) => {
  return newValue;
});
```

☐ วิธีการใช้งาน

1. รับค่าจากอาร์เรย์เก่า
2. ใช้ฟังก์ชันที่กำหนด เพื่อเปลี่ยนแปลงค่าของแต่ละตัว
3. คืนค่าเป็นอาร์เรย์ใหม่

☐ ตัวอย่าง

```
let numbers = [1, 2, 3, 4, 5];  
let doubledNumbers = numbers.map(num => num * 2);  
console.log(doubledNumbers); // [2, 4, 6, 8, 10]
```

2 **filter()** – ใช้กรองข้อมูลในอาร์เรย์

filter() ใช้ สร้างอาร์เรย์ใหม่ โดยเลือกเฉพาะค่าที่ตรงกับเงื่อนไขที่กำหนด

☐ ไวยากรณ์

```
let newArray = oldArray.filter((element, index, array) => {  
    return condition;  
});
```

☐ วิธีการใช้งาน

1. วนลูปผ่านค่าทุกตัวในอาร์เรย์
2. ตรวจสอบเงื่อนไขที่กำหนด
3. ถ้าค่าตรงกับเงื่อนไข ให้เพิ่มลงในอาร์เรย์ใหม่

☐ ตัวอย่าง

```
let numbers = [5, 10, 15, 20, 25];  
let greaterThanTen = numbers.filter(num => num > 10);  
console.log(greaterThanTen); // [15, 20, 25]
```

3 **reduce()** – ใช้รวมค่าทั้งหมดในอาร์เรย์

reduce() ใช้ รวมค่าทั้งหมดในอาร์เรย์ ให้อยู่ในค่าเดียว โดยใช้ฟังก์ชันที่กำหนด

☐ ไวยากรณ์

```
let result = array.reduce((accumulator, currentValue, index, array) => {  
    return newValue;  
}, initialValue);
```

☐ วิธีการใช้งาน

1. ใช้ **ตัวสะสม (accumulator)** เก็บค่าระหว่างการทำซ้ำ
2. ทำงานกับค่าปัจจุบัน (currentValue)
3. คืนค่าผลลัพธ์สุดท้าย

☐ ตัวอย่าง: หาผลรวมของอาร์เรย์ตัวเลข

```
let numbers = [1, 2, 3, 4, 5];  
let sum = numbers.reduce((total, num) => total + num, 0);  
console.log(sum); // 15
```

4 ☐ `forEach()` – ใช้ทำซ้ำแต่ละค่าของอาร์เรย์

`forEach()` ใช้ วนลูปทำงานกับแต่ละค่าของอาร์เรย์ โดยไม่สร้างอาร์เรย์ใหม่

☐ ไวยากรณ์

```
array.forEach((element, index, array) => {
  // คำสั่งที่ต้องการทำซ้ำ
});
```

☐ วิธีการใช้งาน

1. วนลูปผ่านค่าทุกตัวในอาร์เรย์
2. ใช้ฟังก์ชันที่กำหนดเพื่อประมวลผลแต่ละค่า

☐ ตัวอย่าง: แสดงค่าทั้งหมดในอาร์เรย์

```
let numbers = [1, 2, 3, 4, 5];
numbers.forEach(num => console.log(num));
```

☐ เปรียบเทียบเมธอดของอาร์เรย์

เมธอด	ใช้ทำอะไร	เปลี่ยนแปลงอาร์เรย์เดิมหรือไม่?	ผลลัพธ์เป็นอะไร?
<code>map()</code>	สร้างอาร์เรย์ใหม่จากอาร์เรย์เดิม โดยเปลี่ยนค่าทุกตัว	☐ ไม่เปลี่ยนอาร์เรย์เดิม	อาร์เรย์ใหม่
<code>filter()</code>	กรองข้อมูลในอาร์เรย์ตามเงื่อนไขที่กำหนด	☐ ไม่เปลี่ยนอาร์เรย์เดิม	อาร์เรย์ใหม่
<code>reduce()</code>	รวมค่าทั้งหมดในอาร์เรย์เป็นค่าเดียว	☐ ไม่เปลี่ยนอาร์เรย์เดิม	ค่าเดียว (Number, Object, Array ฯลฯ)
<code>forEach()</code>	วนลูปทำงานกับแต่ละค่าของอาร์เรย์	☐ ไม่เปลี่ยนอาร์เรย์เดิม	ไม่มีค่ารีเทิร์น (undefined)

☐ สรุป

- ☐ อาร์เรย์ (Array) คือโครงสร้างข้อมูลที่ใช้เก็บค่าหลายค่าภายในตัวแปรเดียว
- ☐ `map()` ใช้แปลงค่าภายในอาร์เรย์เป็นอาร์เรย์ใหม่
- ☐ `filter()` ใช้คัดกรองข้อมูลที่ตรงเงื่อนไขในอาร์เรย์
- ☐ `reduce()` ใช้รวมค่าทั้งหมดในอาร์เรย์ให้อยู่ในค่าเดียว
- ☐ `forEach()` ใช้ทำงานกับแต่ละค่าของอาร์เรย์โดยไม่สร้างอาร์เรย์ใหม่

ตัวอย่างโค้ด JavaScript แบบเต็มที่ใช้งาน อาร์เรย์ (Array) และเมธอด **map()**, **filter()**, **reduce()**, และ **forEach()** ผังอยู่ใน HTML

□ ฟังก์ชันที่ใช้ในตัวอย่างนี้

- **map()** → คูณตัวเลขในอาร์เรย์ด้วย 2
- **filter()** → กรองตัวเลขที่มากกว่า 10
- **reduce()** → หาผลรวมของตัวเลขทั้งหมด
- **forEach()** → แสดงค่าทั้งหมดในอาร์เรย์

□ ตัวอย่างโค้ด HTML + JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Array Methods Example</title>
</head>
<body>

  <h2>Array Methods in JavaScript</h2>

  <p><strong>Original Array:</strong> <span id="original"></span></p>
  <p><strong>Using map() (Multiply by 2):</strong> <span id="mapResult"></span></p>
  <p><strong>Using filter() (Numbers > 10):</strong> <span id="filterResult"></span></p>
  <p><strong>Using reduce() (Sum of All Numbers):</strong> <span
id="reduceResult"></span></p>
  <p><strong>Using forEach() (Check Console Log)</strong></p>

  <script>
    // สร้างอาร์เรย์ตัวเลข
    let numbers = [5, 10, 15, 20, 25];

    // แสดงค่าอาร์เรย์ดั้งเดิม
    document.getElementById("original").textContent = numbers.join(", ");
```

```
// ใช้ map() - คูณตัวเลขในอาร์เรย์ด้วย 2
let doubledNumbers = numbers.map(num => num * 2);
document.getElementById("mapResult").textContent = doubledNumbers.join(", ");

// ใช้ filter() - กรองตัวเลขที่มากกว่า 10
let greaterThanTen = numbers.filter(num => num > 10);
document.getElementById("filterResult").textContent = greaterThanTen.join(", ");

// ใช้ reduce() - หาผลรวมของตัวเลขทั้งหมด
let sum = numbers.reduce((total, num) => total + num, 0);
document.getElementById("reduceResult").textContent = sum;

// ใช้ forEach() - แสดงค่าทั้งหมดในอาร์เรย์ทาง Console
console.log("Using forEach().");
numbers.forEach(num => console.log(num));
</script>

</body>
</html>
```

☐ คำอธิบายโค้ด

1. สร้างอาร์เรย์ **numbers**
2. `let numbers = [5, 10, 15, 20, 25];`
 - เป็นอาร์เรย์ที่เก็บตัวเลข 5 ตัว
3. ใช้ **map()** เพื่อคูณตัวเลขทุกตัวด้วย 2
4. `let doubledNumbers = numbers.map(num => num * 2);`
 - ตัวเลขใหม่ในอาร์เรย์จะเป็น [10, 20, 30, 40, 50]
5. ใช้ **filter()** เพื่อเลือกเฉพาะตัวเลขที่มากกว่า 10
6. `let greaterThanTen = numbers.filter(num => num > 10);`
 - ผลลัพธ์คือ [15, 20, 25]
7. ใช้ **reduce()** เพื่อหาผลรวมของอาร์เรย์
8. `let sum = numbers.reduce((total, num) => total + num, 0);`
 - ผลรวมของ [5, 10, 15, 20, 25] คือ 75
9. ใช้ **forEach()** เพื่อวนลูปและแสดงค่าทาง **Console**

10. numbers.forEach(num => console.log(num));

- ค่าแต่ละตัวในอาร์เรย์จะถูกพิมพ์ใน Console

☐ ผลลัพธ์ที่ได้เมื่อเปิดในเว็บเบราว์เซอร์

Original Array: 5, 10, 15, 20, 25

Using map() (Multiply by 2): 10, 20, 30, 40, 50

Using filter() (Numbers > 10): 15, 20, 25

Using reduce() (Sum of All Numbers): 75

Using forEach() (Check Console Log)

ใน Console:

Using forEach():

5

10

15

20

25

☐ สรุป

- ☐ **map()** → แปลงค่าของอาร์เรย์เดิมเป็นอาร์เรย์ใหม่
- ☐ **filter()** → กรองค่าตามเงื่อนไข
- ☐ **reduce()** → รวมค่าทั้งหมดในอาร์เรย์
- ☐ **forEach()** → วนลูปแสดงค่าทาง Console

ตัวอย่างการใช้ อาร์เรย์ (Array) และเมธอดของอาร์เรย์ (map, filter, reduce, forEach) เพิ่มอีก 5

ตัวอย่าง พร้อมคำอธิบายละเอียด

☐ ตัวอย่างที่ 1: ใช้ map() แปลงอุณหภูมิจากเซลเซียสเป็นฟาเรนไฮต์

☐ อธิบาย:

- มีอาร์เรย์ของอุณหภูมิในหน่วย เซลเซียส
- ใช้ map() แปลงเป็น ฟาเรนไฮต์ โดยใช้สูตร $F = C * 9/5 + 32$

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```

<meta charset="UTF-8">
<title>แปลงอุณหภูมิ</title>
</head>
<body>

<h2>แปลงอุณหภูมิจากเซลเซียสเป็นฟาเรนไฮต์</h2>
<p><strong>เซลเซียส:</strong> <span id="celsius"></span></p>
<p><strong>ฟาเรนไฮต์:</strong> <span id="fahrenheit"></span></p>

<script>
  let celsiusTemps = [0, 10, 20, 30, 40];
  document.getElementById("celsius").textContent = celsiusTemps.join(", ");

  let fahrenheitTemps = celsiusTemps.map(c => (c * 9/5) + 32);
  document.getElementById("fahrenheit").textContent = fahrenheitTemps.join(", ");
</script>

</body>
</html>

```

☐ ผลลัพธ์:

เซลเซียส: 0, 10, 20, 30, 40

ฟาเรนไฮต์: 32, 50, 68, 86, 104

☐ ตัวอย่างที่ 2: ใช้ `filter()` คัดกรองอายุคนที่สามารถขับรถได้

☐ อธิบาย:

- มีอาร์เรย์ของ อายุคน
- ใช้ `filter()` คัดเฉพาะคนที่มีอายุมากกว่า 18 ปีขึ้นไป

```

<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>คัดกรองอายุ</title>
</head>
<body>

```

```

<h2>คัดกรองอายุคนที่สามารถขับรถได้</h2>
<p><strong>อายุทั้งหมด:</strong> <span id="ages"></span></p>
<p><strong>อายุที่ขับรถได้:</strong> <span id="allowedAges"></span></p>

<script>
  let ages = [12, 16, 18, 20, 25, 30, 15];
  document.getElementById("ages").textContent = ages.join(", ");

  let allowedAges = ages.filter(age => age >= 18);
  document.getElementById("allowedAges").textContent = allowedAges.join(", ");
</script>

</body>
</html>

```

☐ ผลลัพธ์:

อายุทั้งหมด: 12, 16, 18, 20, 25, 30, 15

อายุที่ขับรถได้: 18, 20, 25, 30

☐ ตัวอย่างที่ 3: ใช้ `reduce()` คำนวณค่าเฉลี่ยคะแนนสอบ

☐ อธิบาย:

- มีอาร์เรย์ของ คะแนนสอบ
- ใช้ `reduce()` หาผลรวม แล้วคำนวณค่าเฉลี่ย

```

<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>คำนวณค่าเฉลี่ย</title>
</head>
<body>

  <h2>คำนวณค่าเฉลี่ยคะแนนสอบ</h2>

  <p><strong>คะแนน:</strong> <span id="scores"></span></p>
  <p><strong>ค่าเฉลี่ย:</strong> <span id="average"></span></p>

```

```

<script>
  let scores = [80, 90, 70, 85, 95];
  document.getElementById("scores").textContent = scores.join(", ");

  let total = scores.reduce((sum, score) => sum + score, 0);
  let average = total / scores.length;

  document.getElementById("average").textContent = average.toFixed(2);
</script>

```

```

</body>

```

```

</html>

```

☐ ผลลัพธ์:

คะแนน: 80, 90, 70, 85, 95

ค่าเฉลี่ย: 84.00

☐ ตัวอย่างที่ 4: ใช้ `forEach()` แสดงรายการสินค้า

☐ อธิบาย:

- มีอาร์เรย์ของ สินค้า
- ใช้ `forEach()` วนloopแสดงผลบนหน้าเว็บ

```

<!DOCTYPE html>

```

```

<html lang="th">

```

```

<head>

```

```

  <meta charset="UTF-8">

```

```

  <title>รายการสินค้า</title>

```

```

</head>

```

```

<body>

```

```

  <h2>รายการสินค้า</h2>

```

```

  <ul id="productList"></ul>

```

```

  <script>

```

```

    let products = ["โทรศัพท์", "โน้ตบุ๊ก", "แท็บเล็ต", "สมาร์ทวอตช์"];

```

```

let productList = document.getElementById("productList");
products.forEach(product => {
    let listItem = document.createElement("li");
    listItem.textContent = product;
    productList.appendChild(listItem);
});
</script>

```

```
</body>
```

```
</html>
```

☐ ผลลัพธ์:

รายการสินค้า:

- โทรศัพท์
- โน้ตบุ๊ก
- แท็บเล็ต
- สมาร์ทวอตช์

☐ ตัวอย่างที่ 5: ใช้ `map()`, `filter()`, และ `reduce()` รวมกัน (คำนวณส่วนลดสินค้า)

☐ อธิบาย:

- มีอาร์เรย์ของ ราคาสินค้า
- ใช้ `map()` ลดราคา **10%**
- ใช้ `filter()` คัดเฉพาะสินค้าที่ราคา มากกว่า **100 บาท**
- ใช้ `reduce()` หาผลรวมของราคาสินค้าที่ลดแล้ว

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>คำนวณราคาสินค้า</title>
```

```
</head>
```

```
<body>
```

```
  <h2>คำนวณราคาหลังหักส่วนลด</h2>
```

```
  <p><strong>ราคาสินค้าก่อนลด:</strong> <span id="prices"></span></p>
```

```

<p><strong>ราคาหลังลด:</strong> <span id="discountedPrices"></span></p>
<p><strong>รวมราคาสินค้าหลังลดที่เกิน 100 บาท:</strong> <span
id="totalPrice"></span></p>

<script>
  let prices = [50, 150, 200, 90, 300];
  document.getElementById("prices").textContent = prices.join(", ");

  let discountedPrices = prices.map(price => price * 0.9);
  document.getElementById("discountedPrices").textContent = discountedPrices.map(p =>
p.toFixed(2)).join(", ");

  let filteredPrices = discountedPrices.filter(price => price > 100);

  let totalPrice = filteredPrices.reduce((sum, price) => sum + price, 0);
  document.getElementById("totalPrice").textContent = totalPrice.toFixed(2);
</script>

</body>
</html>

```

☐ ผลลัพธ์:

ราคาสินค้าก่อนลด: 50, 150, 200, 90, 300

ราคาหลังลด: 45.00, 135.00, 180.00, 81.00, 270.00

รวมราคาสินค้าหลังลดที่เกิน 100 บาท: 585.00

☐ สรุป 5 ตัวอย่างเพิ่มเติม

- ☐ map() → แปลงข้อมูล (เช่น ลดราคา, แปลงหน่วย)
- ☐ filter() → คัดกรองข้อมูลตามเงื่อนไข
- ☐ reduce() → คำนวณค่าผลรวม
- ☐ forEach() → วนลูปและแสดงผล

ตัวอย่างเชิงประยุกต์: ระบบจัดการสินค้าออนไลน์

☐ ฟังก์ชันหลักในระบบ:

1. แสดงรายการสินค้า – ใช้ forEach()

2. ค้นหาสินค้าที่มีราคาเกิน 500 บาท – ใช้ filter()
3. คำนวณราคาสินค้าหลังหักส่วนลด 10% – ใช้ map()
4. หาผลรวมราคาสินค้าทั้งหมดที่มีในร้าน – ใช้ reduce()
5. เพิ่มสินค้าใหม่ลงไปรายการสินค้า

□ ตัวอย่างโค้ด HTML + JavaScript (ฝัง JS ใน HTML)

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>ระบบจัดการสินค้าออนไลน์</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .container { max-width: 600px; margin: auto; }
    button { margin-top: 10px; padding: 10px; cursor: pointer; }
    ul { list-style-type: none; padding: 0; }
    li { background: #f8f8f8; margin: 5px 0; padding: 10px; border-radius: 5px; }
  </style>
</head>
<body>

<div class="container">
  <h2>□ รายการสินค้า</h2>
  <ul id="productList"></ul>

  <h3>□ ค้นหาสินค้าที่มีราคาเกิน 500 บาท</h3>
  <button onclick="filterExpensiveProducts()">แสดงสินค้าแพง</button>
  <ul id="expensiveProducts"></ul>

  <h3>□ คำนวณราคาหลังลด 10%</h3>
  <button onclick="applyDiscount()">แสดงราคาหลังลด</button>
  <ul id="discountedPrices"></ul>

  <h3>□ คำนวณผลรวมราคาสินค้าทั้งหมด</h3>
```

```
<button onclick="calculateTotal()">คำนวณผลรวม</button>
<p id="totalPrice"></p>

<h3>□ เพิ่มสินค้าใหม่</h3>
<input type="text" id="newProductName" placeholder="ชื่อสินค้า">
<input type="number" id="newProductPrice" placeholder="ราคา">
<button onclick="addProduct()">เพิ่มสินค้า</button>
</div>

<script>
// □ อาร์เรย์รายการสินค้า (ชื่อและราคา)
let products = [
  { name: "โทรศัพท์", price: 8000 },
  { name: "โน้ตบุ๊ก", price: 25000 },
  { name: "หูฟัง", price: 1500 },
  { name: "กระเป๋า", price: 499 },
  { name: "รองเท้า", price: 1200 }
];

// □ ฟังก์ชันแสดงรายการสินค้า
function displayProducts() {
  let productList = document.getElementById("productList");
  productList.innerHTML = "";
  products.forEach(product => {
    let li = document.createElement("li");
    li.textContent = `${product.name} - ${product.price} บาท`;
    productList.appendChild(li);
  });
}

// □ ฟังก์ชันค้นหาสินค้าราคาเกิน 500 บาท
function filterExpensiveProducts() {
  let expensiveList = document.getElementById("expensiveProducts");
  expensiveList.innerHTML = "";
```

```
let expensiveProducts = products.filter(p => p.price > 500);
expensiveProducts.forEach(product => {
  let li = document.createElement("li");
  li.textContent = `${product.name} - ${product.price} บาท`;
  expensiveList.appendChild(li);
});
}
```

// ☐ ฟังก์ชันคำนวณราคาหลังลด 10%

```
function applyDiscount() {
  let discountedList = document.getElementById("discountedPrices");
  discountedList.innerHTML = "";
  let discountedProducts = products.map(p => ({
    name: p.name,
    price: (p.price * 0.9).toFixed(2)
  }));
  discountedProducts.forEach(product => {
    let li = document.createElement("li");
    li.textContent = `${product.name} - ${product.price} บาท`;
    discountedList.appendChild(li);
  });
}
```

// ☐ ฟังก์ชันคำนวณผลรวมราคาสินค้าทั้งหมด

```
function calculateTotal() {
  let total = products.reduce((sum, p) => sum + p.price, 0);
  document.getElementById("totalPrice").textContent = `
☐ ผลรวมราคา: ${total} บาท`;
}
```

// ☐ ฟังก์ชันเพิ่มสินค้าใหม่

```
function addProduct() {
  let name = document.getElementById("newProductName").value;
  let price = parseFloat(document.getElementById("newProductPrice").value);
  if (name && price > 0) {
```

```

        products.push({ name, price });
        displayProducts(); // อัปเดตรายการสินค้า
        document.getElementById("newProductName").value = "";
        document.getElementById("newProductPrice").value = "";
    } else {
        alert("กรุณกรอกชื่อและราคาสินค้าที่ถูกต้อง");
    }
}

// ☐ แสดงรายการสินค้าตั้งแต่เริ่มต้น
displayProducts();
</script>

</body>
</html>

```

☐ คำอธิบายโค้ด

1. แสดงรายการสินค้า
 - ใช้ `forEach()` วนลูปแสดงสินค้าใน `<ul id="productList">`
2. ค้นหาสินค้าที่มีราคาเกิน 500 บาท
 - ใช้ `filter()` คัดเฉพาะสินค้าราคาเกิน 500 บาท แล้วแสดงใน `<ul id="expensiveProducts">`
3. คำนวณราคาหลังหักส่วนลด 10%
 - ใช้ `map()` ลดราคา 10% และแสดงผลใน `<ul id="discountedPrices">`
4. หาผลรวมราคาสินค้าทั้งหมด
 - ใช้ `reduce()` คำนวณผลรวม แล้วแสดงใน `<p id="totalPrice">`
5. เพิ่มสินค้าใหม่
 - รับค่าจาก `<input>` และเพิ่มสินค้าใหม่ในอาร์เรย์
 - เรียก `displayProducts()` เพื่ออัปเดตรายการสินค้า

☐ ตัวอย่างการทำงาน

- ☐ รายการสินค้าแสดงผลบนหน้าเว็บ
- ☐ ปุ่ม แสดงสินค้าแพง จะแสดงสินค้า ที่ราคาเกิน 500 บาท
- ☐ ปุ่ม แสดงราคาหลังลด แสดงราคาสินค้า หลังลด 10%

- ☐ ปุ่ม คำนวณผลรวม แสดง ผลรวมราคาสินค้าทั้งหมด
- ☐ ปุ่ม เพิ่มสินค้า สามารถเพิ่มสินค้าใหม่เข้าไปในรายการได้

☐ สรุปการประยุกต์

- ✓ ☐ ใช้ **forEach()** แสดงรายการสินค้า
- ✓ ☐ ใช้ **filter()** คัดกรองสินค้าที่มีราคาแพง
- ✓ ☐ ใช้ **map()** ลดราคาสินค้า
- ✓ ☐ ใช้ **reduce()** คำนวณผลรวมราคา
- ✓ ☐ ใช้ **DOM Manipulation** เพื่ออัปเดตข้อมูลแบบไดนามิก

☐ ☐ นี่คือนตัวอย่างจริงของการใช้ **map()**, **filter()**, **reduce()**, และ **forEach()** ในระบบร้านค้าออนไลน์ ☐

อ็อบเจกต์ (Object) และการเข้าถึงข้อมูลใน JavaScript

☐ 1. ความหมายของ Object

Object เป็นโครงสร้างข้อมูลที่ใช้เก็บค่าต่าง ๆ ในรูปแบบของ "คู่คีย์-ค่า" (**key-value pairs**) ซึ่งช่วยให้สามารถจัดการข้อมูลที่ซับซ้อนมากกว่าการใช้ตัวแปรแบบดั้งเดิม

☐ 2. ความจำเป็นของ Object

1. จัดการข้อมูลที่มีโครงสร้างชัดเจน เช่น รายละเอียดสินค้า, ข้อมูลผู้ใช้
2. เก็บค่าที่เกี่ยวข้องกันไว้ในที่เดียว ทำให้จัดการได้ง่าย
3. สามารถเข้าถึงและปรับแต่งข้อมูลได้อย่างสะดวก

☐ 3. ไวยากรณ์ของ Object

```
let objectName = {
  key1: value1,
  key2: value2,
  key3: value3
};
```

☐ ตัวอย่าง:

```
let user = {
  name: "สมชาย",
  age: 25,
```

```
isMember: true  
};
```

□ 4. วิธีการเข้าถึงข้อมูลใน Object

มี 2 วิธีหลักคือ

1. Dot Notation (.)

2. Bracket Notation ([])

```
console.log(user.name);    // "สมชาย" (ใช้ Dot Notation)  
console.log(user["age"]);  // 25 (ใช้ Bracket Notation)
```

□ 5. วิธีการเพิ่ม/แก้ไขค่าภายใน Object

สามารถเพิ่มหรือแก้ไขค่าโดยใช้ =

```
user.phone = "0812345678"; // เพิ่มข้อมูลใหม่  
user.age = 26;             // แก้ไขค่า  
console.log(user);
```

□ 6. วิธีการลบข้อมูลใน Object

ใช้ delete เพื่อลบค่าภายใน Object

```
delete user.isMember;  
console.log(user);
```

□ 7. การใช้ Object ร่วมกับฟังก์ชัน

สามารถส่ง Object เข้าไปในฟังก์ชันเพื่อประมวลผล

```
function showUserInfo(user) {  
    return `ชื่อ: ${user.name}, อายุ: ${user.age} ปี`;  
}  
console.log(showUserInfo(user));
```

□ 8. การวนลูป Object ด้วย for...in

ใช้ for...in เพื่อวนลูปดึงข้อมูลใน Object

```
for (let key in user) {  
    console.log(`${key}: ${user[key]}`);  
}
```

□ ตัวอย่างโค้ดเชิงประยุกต์: ระบบจัดการข้อมูลนักเรียน

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>ระบบนักเรียน</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .container { max-width: 500px; margin: auto; }
    button { margin-top: 10px; padding: 10px; cursor: pointer; }
  </style>
</head>
<body>

<div class="container">
  <h2>□ ระบบจัดการข้อมูลนักเรียน</h2>
  <p id="studentInfo"></p>
  <button onclick="updateAge()">เพิ่มอายุ +1 ปี</button>
</div>

<script>
  let student = {
    name: "ปริญญา",
    age: 20,
    grade: "A"
  };

  function displayStudent() {
    document.getElementById("studentInfo").innerHTML =
      `□ □ ชื่อ: ${student.name} | อายุ: ${student.age} ปี | เกรด: ${student.grade}`;
  }

  function updateAge() {
    student.age += 1;
  }
</script>
</body>
</html>
```

```
displayStudent();
}
```

```
displayStudent();
</script>
```

```
</body>
</html>
```

☐ สรุป

- ✓ ☐ **Object** คือโครงสร้างข้อมูลที่ใช้เก็บ "คู่คีย์-ค่า"
- ✓ ☐ สามารถเข้าถึงข้อมูลได้ด้วย `.` หรือ `[]`
- ✓ ☐ สามารถเพิ่ม, แก้ไข และลบค่าได้
- ✓ ☐ สามารถใช้ **Object** ร่วมกับฟังก์ชันและวนลูป `for...in` ได้

☐ ตัวอย่างเพิ่มเติมของ **Object** ใน JavaScript

☐ ตัวอย่างที่ 1: ระบบลงทะเบียนผู้ใช้งาน

☐ อธิบาย:

- ใช้ Object เก็บข้อมูลผู้ใช้ เช่น ชื่อ อีเมล และสถานะ
- ใช้ฟังก์ชันเพื่อเปลี่ยนสถานะผู้ใช้

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>ระบบลงทะเบียนผู้ใช้</title>
```

```
</head>
```

```
<body>
```

```
<h2> ☐ ข้อมูลผู้ใช้งาน</h2>
```

```
<p id="userInfo"></p>
```

```
<button onclick="activateUser()"> ☐ เปิดใช้งาน</button>
```

```
<script>
  let user = {
    name: "อานันต์",
    email: "anan@example.com",
    isActive: false
  };

  function displayUser() {
    document.getElementById("userInfo").innerHTML =
      `❑❑ ชื่อ: ${user.name} | ❑ อีเมล: ${user.email} | สถานะ: ${user.isActive ? "เปิดใช้
งาน" : "ปิดใช้งาน"}`;
  }

  function activateUser() {
    user.isActive = true;
    displayUser();
  }

  displayUser();
</script>

</body>
</html>
```

-
- ❑ ตัวอย่างที่ 2: คำนวณราคาสินค้าหลังจากใช้ส่วนลด
 - ❑ อธิบาย:
 - ใช้ Object เก็บข้อมูลสินค้า เช่น ชื่อ ราคา และส่วนลด
 - ใช้ฟังก์ชันคำนวณราคาสุทธิ

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>คำนวณราคาสินค้า</title>
```

```
</head>
<body>

<h2>☐ คำนวณราคาสินค้า</h2>
<p id="productInfo"></p>

<script>
  let product = {
    name: "โน้ตบุ๊ก",
    price: 25000,
    discount: 10 // ส่วนลดเป็นเปอร์เซ็นต์
  };

  function calculateDiscountedPrice() {
    return product.price - (product.price * (product.discount / 100));
  }

  document.getElementById("productInfo").innerHTML =
    `☐ สินค้า: ${product.name} | ☐ ราคาปกติ: ${product.price} บาท | ☐ ราคาหลังหักส่วนลด:
    ${calculateDiscountedPrice()} บาท`;
</script>

</body>
</html>
```

☐ ตัวอย่างที่ 3: ตรวจสอบคะแนนสอบและแสดงเกรด

☐ อธิบาย:

- ใช้ Object เก็บข้อมูลนักเรียนและคะแนน
- ใช้เงื่อนไขในการตรวจสอบและให้เกรด

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>ตรวจสอบเกรด</title>
```

```
</head>
<body>

<h2>❑ ตรวจสอบเกรดของนักเรียน</h2>
<p id="gradeInfo"></p>

<script>
  let student = {
    name: "สุธี",
    score: 82
  };

  function getGrade(score) {
    if (score >= 80) return "A";
    else if (score >= 70) return "B";
    else if (score >= 60) return "C";
    else if (score >= 50) return "D";
    else return "F";
  }

  document.getElementById("gradeInfo").innerHTML =
    `❑❑ นักเรียน: ${student.name} | ❑ คะแนน: ${student.score} | ❑ เกรด:
    ${getGrade(student.score)}`;
</script>

</body>
</html>
```

❑ ตัวอย่างที่ 4: ระบบจองโรงแรม

❑ อธิบาย:

- ใช้ Object เก็บข้อมูลโรงแรม
- ใช้ฟังก์ชันในการเปลี่ยนสถานะห้องพัก

```
<!DOCTYPE html>
<html lang="th">
```

```
<head>
  <meta charset="UTF-8">
  <title>ระบบจองโรงแรม</title>
</head>
<body>

<h2>□ ระบบจองโรงแรม</h2>
<p id="hotelInfo"></p>
<button onclick="bookRoom()">□ จองห้องพัก</button>

<script>
  let hotel = {
    name: "Luxury Hotel",
    availableRooms: 5
  };

  function bookRoom() {
    if (hotel.availableRooms > 0) {
      hotel.availableRooms -= 1;
      alert("□ จองห้องพักสำเร็จ!");
    } else {
      alert("□ ห้องพักรับเต็มแล้ว!");
    }
    displayHotel();
  }

  function displayHotel() {
    document.getElementById("hotelInfo").innerHTML =
      `□ โรงแรม: ${hotel.name} | ห้องว่าง: ${hotel.availableRooms} ห้อง`;
  }

  displayHotel();
</script>
```

```
</body>
```

```
</html>
```

☐ ตัวอย่างที่ 5: จัดการข้อมูลพนักงาน

☐ อธิบาย:

- ใช้ Object เก็บข้อมูลพนักงาน
- ใช้ฟังก์ชันเพิ่มเงินเดือน

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>จัดการข้อมูลพนักงาน</title>
```

```
</head>
```

```
<body>
```

```
<h2> ☐ ☐ ข้อมูลพนักงาน</h2>
```

```
<p id="employeeInfo"></p>
```

```
<button onclick="increaseSalary()"> ☐ เพิ่มเงินเดือน</button>
```

```
<script>
```

```
  let employee = {
```

```
    name: "กิตติ",
```

```
    position: "นักพัฒนาเว็บ",
```

```
    salary: 30000
```

```
  };
```

```
  function increaseSalary() {
```

```
    employee.salary += 5000;
```

```
    alert(` ☐ เงินเดือนเพิ่มเป็น ${employee.salary} บาทแล้ว!`);
```

```
    displayEmployee();
```

```
  }
```

```
  function displayEmployee() {
```

```
    document.getElementById("employeeInfo").innerHTML =
```

```

        `□ ชื่อ: ${employee.name} | □ ตำแหน่ง: ${employee.position} | □ เงินเดือน:
        ${employee.salary} บาท`;
    }

    displayEmployee();
</script>

</body>
</html>

```

- สรุป
 - **Object** ใช้เก็บข้อมูลที่เกี่ยวข้องกัน เช่น ผู้ใช้, สินค้า, พนักงาน
 - สามารถเข้าถึงข้อมูลได้ด้วย . และ []
 - สามารถใช้ **Object** ร่วมกับฟังก์ชันเพื่อประมวลผลข้อมูล
 - สามารถอัปเดตหรือลบค่าได้ง่าย
- ตัวอย่างการประยุกต์ใช้ **Array + Object** ใน JavaScript (5 ตัวอย่าง)

- ตัวอย่างที่ 1: ระบบจัดการสินค้าคงคลัง
- อธิบาย:
 - ใช้ **Array** เก็บรายการสินค้าแต่ละตัวเป็น **Object**
 - ใช้ฟังก์ชันเพื่อแสดงข้อมูลและอัปเดตสต็อกสินค้า

```

<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <title>□ ระบบจัดการสินค้าคงคลัง</title>
</head>
<body>

<h2>□ รายการสินค้า</h2>
<ul id="productList"></ul>
<button onclick="sellProduct(0)">□ ขาย iPhone</button>
<button onclick="sellProduct(1)">□ ขาย MacBook</button>

```

```
<script>
  let products = [
    { name: "iPhone", price: 30000, stock: 10 },
    { name: "MacBook", price: 50000, stock: 5 }
  ];

  function displayProducts() {
    let list = "";
    products.forEach((p, index) => {
      list += `<li>${p.name} - ราคา: ${p.price} บาท | คงเหลือ: ${p.stock} ชิ้น</li>`;
    });
    document.getElementById("productList").innerHTML = list;
  }

  function sellProduct(index) {
    if (products[index].stock > 0) {
      products[index].stock -= 1;
      alert(`☑ ขาย ${products[index].name} สำเร็จ!`);
    } else {
      alert(`☐ ${products[index].name} หมดสต็อก!`);
    }
    displayProducts();
  }

  displayProducts();
</script>

</body>
</html>
```

-
- ☐ ตัวอย่างที่ 2: ระบบจัดการหักเรียนและคำนวณเกรด
- ☐ อธิบาย:
- ใช้ **Array** เก็บนักเรียนแต่ละคนเป็น **Object**

- ใช้ฟังก์ชันในการคำนวณเกรด

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>□ ระบบคำนวณเกรด</title>
```

```
</head>
```

```
<body>
```

```
<h2>□ ข้อมูลนักเรียน</h2>
```

```
<ul id="studentList"></ul>
```

```
<script>
```

```
  let students = [
```

```
    { name: "สมชาย", score: 85 },
```

```
    { name: "มานี", score: 72 },
```

```
    { name: "วีระ", score: 60 }
```

```
  ];
```

```
  function getGrade(score) {
```

```
    if (score >= 80) return "A";
```

```
    else if (score >= 70) return "B";
```

```
    else if (score >= 60) return "C";
```

```
    else return "F";
```

```
  }
```

```
  function displayStudents() {
```

```
    let list = "";
```

```
    students.forEach(student => {
```

```
      list += `<li>${student.name} - คะแนน: ${student.score} | เกรด:
```

```
      ${getGrade(student.score)}</li>`;
```

```
    });
```

```
    document.getElementById("studentList").innerHTML = list;
```

```
  }
```

```
displayStudents();
</script>

</body>
</html>
```

☐ ตัวอย่างที่ 3: ระบบแสดงรายการหนังสือ

☐ อธิบาย:

- ใช้ **Array** เก็บข้อมูลหนังสือ
- ใช้ฟังก์ชัน `map()` แปลงข้อมูลเป็น HTML

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>☐ รายการหนังสือ</title>
</head>
<body>

<h2>☐ รายการหนังสือ</h2>
<ul id="bookList"></ul>

<script>
  let books = [
    { title: "JavaScript ขั้นเทพ", author: "สมศักดิ์", year: 2020 },
    { title: "Python ฉบับสมบูรณ์", author: "มานพ", year: 2019 },
    { title: "React JS สำหรับมือใหม่", author: "กิตติ", year: 2021 }
  ];

  document.getElementById("bookList").innerHTML = books.map(
    book => `<li>${book.title} - เขียนโดย ${book.author} (${book.year})</li>`
  ).join("");
</script>
```

```
</body>
```

```
</html>
```

□ ตัวอย่างที่ 4: ระบบคำนวณเงินเดือนพนักงาน

□ อธิบาย:

- ใช้ **Array** เก็บข้อมูลพนักงานแต่ละคน
- ใช้ฟังก์ชัน `reduce()` คำนวณเงินเดือนรวม

```
<!DOCTYPE html>
```

```
<html lang="th">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>□ คำนวณเงินเดือนพนักงาน</title>
```

```
</head>
```

```
<body>
```

```
<h2>□ รายละเอียดพนักงาน</h2>
```

```
<ul id="employeeList"></ul>
```

```
<p><strong>□ เงินเดือนรวม:</strong> <span id="totalSalary"></span> บาท</p>
```

```
<script>
```

```
  let employees = [
```

```
    { name: "ก้อง", position: "นักพัฒนาเว็บ", salary: 35000 },
```

```
    { name: "แอน", position: "นักออกแบบ UX/UI", salary: 40000 },
```

```
    { name: "สมชาย", position: "นักวิเคราะห์ข้อมูล", salary: 45000 }
  ];
```

```
  function displayEmployees() {
```

```
    let list = employees.map(emp =>
```

```
      `<li>□ ${emp.name} - ตำแหน่ง: ${emp.position} | เงินเดือน: ${emp.salary} บาท</li>`
```

```
    ).join("");
```

```
    document.getElementById("employeeList").innerHTML = list;
```

```
  }
```

```
  function calculateTotalSalary() {
```

```

        return employees.reduce((sum, emp) => sum + emp.salary, 0);
    }

    displayEmployees();

    document.getElementById("totalSalary").innerText = calculateTotalSalary();
</script>

</body>
</html>

```

☐ ตัวอย่างที่ 5: ระบบกรองสินค้าตามประเภท

☐ อธิบาย:

- ใช้ **Array** เก็บสินค้าแต่ละรายการ
- ใช้ฟังก์ชัน `filter()` แสดงสินค้าเฉพาะประเภทที่ต้องการ

```

<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <title>☐ ระบบกรองสินค้าตามประเภท</title>
</head>
<body>

<h2>☐ รายการสินค้า</h2>
<button onclick="showCategory('มือถือ')">☐ แสดงมือถือ</button>
<button onclick="showCategory('แล็ปท็อป')">☐ แสดงแล็ปท็อป</button>
<ul id="filteredProducts"></ul>

<script>
    let products = [
        { name: "iPhone 13", category: "มือถือ", price: 29000 },
        { name: "Samsung Galaxy S22", category: "มือถือ", price: 27000 },
        { name: "MacBook Pro", category: "แล็ปท็อป", price: 65000 },
        { name: "Dell XPS 13", category: "แล็ปท็อป", price: 55000 }
    ];

```

```
function showCategory(category) {
  let filtered = products.filter(p => p.category === category);
  document.getElementById("filteredProducts").innerHTML = filtered.map(
    p => `<li> ☐ ${p.name} - ราคา: ${p.price} บาท</li>`
  ).join("");
}
</script>

</body>
</html>
```

☐ สรุป

☐ **Array + Object** ใช้ร่วมกันได้ดีเมื่อจัดการข้อมูลแบบรายการ เช่น

- ☐ สินค้า
- ☐ หนังสือ
- ☐ นักเรียน
- ☐ พนักงาน

☐ ใช้เมธอดของ **Array** เช่น **forEach()**, **map()**, **filter()**, **reduce()** เพื่อช่วยจัดการและคำนวณข้อมูล

สรุป

ใน JavaScript การจัดการข้อมูลหลักมักใช้ อาร์เรย์ (**Array**) และ อ็อบเจกต์ (**Object**) เป็นเครื่องมือพื้นฐาน โดยอาร์เรย์เหมาะสำหรับเก็บข้อมูลหลายค่าในลำดับ เช่น รายการสินค้า หรือรายชื่อนักเรียน ซึ่งสามารถเข้าถึงได้ผ่าน index เริ่มจาก 0 เมธอดสำคัญของอาร์เรย์ ได้แก่ **map()** ที่แปลงข้อมูลเป็นอาร์เรย์ใหม่, **filter()** ที่คัดเลือกข้อมูลตามเงื่อนไข, **reduce()** ที่รวมค่าทั้งหมดให้เป็นค่าเดียว เช่น ผลรวม และ **forEach()** ที่ใช้วนลูปโดยไม่คืนค่า การใช้เมธอดเหล่านี้ช่วยให้จัดการข้อมูลในอาร์เรย์ได้ง่ายและมีประสิทธิภาพ

ในขณะที่อ็อบเจกต์ (**Object**) ใช้เก็บข้อมูลแบบมีโครงสร้างในรูปแบบ **key-value pairs** ซึ่งเหมาะสำหรับข้อมูลที่มีรายละเอียดหลายด้าน เช่น ข้อมูลผู้ใช้หรือรายละเอียดสินค้า เราสามารถเข้าถึงค่าต่าง ๆ ภายในอ็อบเจกต์ได้หลายวิธี เช่น **dot notation** (**user.name**), **bracket notation** (**user["name"]**), หรือใช้ **object destructuring** (**const { name } = user**) นอกจากนี้ยังสามารถใช้เมธอดอย่าง **Object.keys()**, **Object.values()**, และ **Object.entries()** เพื่อดึงข้อมูลในรูปแบบอาร์เรย์ออกมาจัดการเพิ่มเติม การเข้าใจและใช้งานอาร์เรย์ร่วมกับอ็อบเจกต์อย่างถูกต้องจึงเป็นพื้นฐานสำคัญในการพัฒนาโปรแกรมด้วย JavaScript อย่างมีประสิทธิภาพ.